GLOWLOGICS

PROJECT SUBMISSION

CYBERSECURITY AND ETHICAL HACKING

PROJECT TITLE

PHISHING EMAIL SIMULATION

A controlled awareness-training demonstration using fictional messages and simulated responses

Project Submitted by:
SHIVAANI SHRE SR
Email: shivaani.050@gmail.com

Submitted for internship project evaluation

1. Abstract

Phishing is a social-engineering technique in which messages imitate trusted senders to encourage unsafe actions. This project demonstrates phishing awareness in a controlled, educational environment. It uses fictional email examples and a local simulation page that records only a participant's selected response, such as “Report suspicious email” or “Mark as suspicious.” It does not send real phishing emails, collect passwords, or impersonate a real organization. The aim is to help learners identify warning signs and understand prevention measures.

2. Objectives

- Explain common phishing indicators, including urgency, unexpected attachments, suspicious links, and mismatched sender details.
- Create fictional sample messages for security-awareness training.
- Demonstrate a safe local response-tracking simulation without collecting credentials or personal data.
- Summarize learning outcomes and recommend preventive actions.

3. Scope and Ethical Controls

- Use only fictional names, domains, and message content.
- Run the demonstration locally or in a training environment with participant consent.
- Do not send messages to unsuspecting recipients or use real company branding to deceive people.
- Never request, record, or store passwords, one-time passcodes, or sensitive personal information.
- Use the results for education and improvement, not for shaming individual participants.

4. Tools and Technologies

Technology	Purpose
Python 3	Runs a small local training simulation.
Flask (optional)	Can provide a local web interface if installed.
HTML/CSS	Displays fictional sample messages and response options.
CSV / text file	Stores aggregate, non-sensitive training choices.

5. System Design

- **Step 1 — Prepare:** The trainer selects a fictional email scenario and explains the exercise.
- **Step 2 — Present:** The local page displays the sample message and asks the learner to identify warning signs.
- **Step 3 — Respond:** The learner chooses an action: “Report suspicious,” “Inspect sender,” or “Treat as legitimate.”
- **Step 4 — Feedback:** The page explains which indicators were present and provides safe handling advice.
- **Step 5 — Review:** The trainer summarizes aggregate responses and discusses how to report suspicious emails.

6. Sample Python Program

```
"""
Phishing Awareness Simulation
Educational demo only: no emails are sent and no credentials are collected.
"""

from datetime import datetime
import csv
```

```
SCENARIOS = [
    {
        "subject": "Action requested: verify your training account",
        "sender": "training-alert@example.test",
        "message": (
            "This fictional message asks you to act quickly and follow an "
            "unexpected link. Inspect the sender and verify requests through "
            "a trusted channel before taking action."
        ),
        "warning_signs": [
            "Unexpected request",
            "Urgent wording",
            "Link should be verified independently"
        ]
    }
]

def run_simulation():
    print("PHISHING AWARENESS SIMULATION")
    print("Training only: no messages are sent; do not enter passwords.")
    scenario = SCENARIOS[0]

    print("\\nFrom:", scenario["sender"])
    print("Subject:", scenario["subject"])
    print("\\nMessage:", scenario["message"])
    print("\\nChoose your response:")
    print("1. Report as suspicious")
    print("2. Inspect sender and verify independently")
    print("3. Treat as legitimate")

    choice = input("Enter 1, 2, or 3: ").strip()
    actions = {
        "1": "Reported as suspicious",
        "2": "Selected independent verification",
        "3": "Treated as legitimate"
    }
    result = actions.get(choice, "Invalid response")

    # Save only timestamp and selected training action; no personal data.
    with open("awareness_results.csv", "a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow([datetime.now().isoformat(timespec="seconds"), result])

    print("\\nYour response:", result)
    print("Learning feedback:")
    for sign in scenario["warning_signs"]:
        print("-", sign)
    print("\\nNever click unexpected links or provide credentials from an email link.")
    print("Results saved to awareness_results.csv")

if __name__ == "__main__":
    run_simulation()
```

7. How to Run the Program

- Install Python 3 if it is not already installed.
- Save the code as **phishing_awareness.py**.
- Open a terminal in the file's folder and run **python phishing_awareness.py** (or **python3 phishing_awareness.py**).
- Choose one of the three training responses when prompted.
- The program creates **awareness_results.csv** with a timestamp and response choice only.

8. Illustrative Output

```
PHISHING AWARENESS SIMULATION
Training only: no messages are sent; do not enter passwords.

From: training-alert@example.test
Subject: Action requested: verify your training account

Choose your response:
1. Report as suspicious
2. Inspect sender and verify independently
3. Treat as legitimate
Enter 1, 2, or 3: 1
```

```
Your response: Reported as suspicious
Learning feedback:
- Unexpected request
- Urgent wording
- Link should be verified independently

Results saved to awareness_results.csv
```

The output above is an example. The actual response depends on the option selected by the learner.

9. Identifying Phishing Indicators

- **Sender mismatch:** The display name and email address may not match.
- **Unexpected urgency:** The message pressures the recipient to act immediately.
- **Suspicious links:** The visible text may differ from the actual destination.
- **Unexpected attachments:** Files should be verified before opening.
- **Requests for secrets:** Passwords and one-time codes should never be supplied through an unsolicited email link.

10. Results and Discussion

The prototype demonstrates how an awareness exercise can present a fictional message, collect a non-sensitive training choice, and provide immediate feedback. The CSV file supports a basic review of response patterns. It is not a production phishing platform and does not measure real-world security by itself. A real training program should be approved by the organization and should communicate clear rules for privacy, consent, and data retention.

11. Preventive Measures

- Verify unexpected requests using a known, trusted contact method.
- Check the full sender address and link destination before acting.
- Avoid opening unexpected attachments.
- Use multi-factor authentication and a password manager.
- Report suspicious emails to the organization's security or IT team.
- Keep devices, browsers, and email software updated.

12. Limitations and Future Enhancements

Limitations: The program includes one fixed scenario, runs in a terminal, and records only a simple response. It does not send email, authenticate users, or evaluate all phishing techniques.

Future enhancements: Add more fictional scenarios, a local graphical interface, accessibility support, aggregate dashboards, and a trainer guide. Any real-world deployment should be approved and privacy-reviewed.

13. Conclusion

The Phishing Email Simulation project introduces social-engineering awareness through a controlled, local exercise. It helps learners practice identifying suspicious messages, choosing safe responses, and understanding prevention strategies while avoiding real phishing emails and the collection of sensitive information.

14. References

- CISA, Recognize and Report Phishing: https://www.cisa.gov/secure-our-world/recognize-and-report-phishing
- OWASP, Security Awareness: https://owasp.org/
- Python documentation: https://docs.python.org/3/
- Python csv module documentation: https://docs.python.org/3/library/csv.html